

Session 3 Assignment — Contract Implementation

Course: Platform-Based Software Engineering **Unit of work:** Group (4 people), accompanied by an individual contribution statement **Weight:** Part of 15% of the lab grade. The resulting code becomes the basis for A2 — Backend Service (15% of the final grade), due at Session 11

o. General provisions

1. **The contract is the reference, and the implementation follows it.** If the implementation does not match a correct contract, the implementation is fixed. If the contract itself is wrong, the contract is revised deliberately, recorded in `CHANGELOG.md` with the reason, and its version is bumped according to the compatibility policy from the Session 2 assignment. Changing `openapi.yaml` merely to make a failing test pass is graded FAIL for Part A. This rule applies because eight consumer classes are built against the same document, and at Session 7 other groups build clients based on that document without communicating with its author. Undisclosed changes will not be detected until that point.
 2. **The mock server from Session 2 keeps running throughout this session.** The implementation replaces the mock one operation at a time, not all at once. The member doing testing must be able to switch between the mock and the service by changing only the base URL. An operation that works against the service but not against the mock indicates the implementation has drifted from the specification.
 3. **Each member commits using their own Git identity.** This is verified with `git log --format='%an' | sort -u` in the `service/` directory.
 4. **No configuration values or credentials exist in the source code.** Any credential that has ever been committed must be rotated, not deleted, because the repository history still holds that value after the file is changed.
 5. **There is no authentication yet at this session.** The resulting service can be accessed by anyone who knows the URL. This is intentional and is fixed at Session 4. Consequence: do not put any personal data into the database this session.
-

Part A — A Running Service That Matches the Contract

Due: end of the Session 3 lab · **Repository tag:** `13` · with the deployment URL

Work is done against each group's own domain and against the `openapi.yaml` validated at Session 2. The CANTEEN case study in the course material serves only as an example of form.

A.1 Service directory structure

The following structure is required and must be used. It continues the repository skeleton created at Session 2; `openapi.yaml` stays at the repository root because clients from Session 5 through 12 read it from that location.

<code>openapi.yaml</code>	contract – stays at the root, above the service
<code>CHANGELOG.md</code>	
<code>service/</code>	
<code>.env.example</code>	every variable the service needs
<code>db/</code>	
<code>schema.sql</code>	all CREATE TABLE statements, runnable from an empty database
<code>seed.sql</code>	a small amount of sample data for demonstration
<code>src/</code>	
<code>app.js</code>	assembly: routes, error handler, config checks
<code>routes/</code>	
<code><resource>.js</code>	one file per resource; routes plus their handlers
<code>schemas/</code>	
<code><resource>.js</code>	validation rules, derived from the contract
<code>store/</code>	
<code><resource>.js</code>	the only place SQL is written
<code>representations/</code>	
<code><resource>.js</code>	stored row → response shape from the contract
<code>problem.js</code>	one failure shape for the entire API
<code>tests/</code>	
<code>contract/</code>	tests checking the service's conformance to <code>openapi.yaml</code>

File names follow the language and framework the cohort agreed on at Session 1. What is graded is the separation of responsibilities, not the file extensions.

Directory	Responsibility	Reason for the separation
<code>routes/</code>	Binds a method and path to a single function, then runs the five parts of an operation in order.	One file per resource lets every operation on that resource be read in one place.
<code>schemas/</code>	Validation rules copied from <code>parameters</code> and <code>requestBody</code> in the contract.	Validation scattered across handlers causes the same rule to be rewritten in a different shape in every operation.
<code>store/</code>	The only layer that runs database queries.	The handler can be read without reading SQL, and the handler can be tested by swapping this layer.
<code>representations/</code>	The only layer that decides which fields the caller is allowed to see.	Renaming a column becomes a database change, not a breaking interface change for clients.
<code>problem.js</code>	One function that produces every failure response.	The error shape cannot diverge between operations, and adding a field to that shape is a single edit.

Rule on handlers that mix responsibilities. A function that builds SQL and also assembles the response JSON does not satisfy the structural requirement, because the two cannot be changed independently. This is the most common cause of the internal column leaks described in section A.4.

A.2 Five parts of every operation

Every operation in `openapi.yaml` is implemented as five parts in a fixed order. This structure is the basis for grading completeness.

Part	Question it answers	Source in <code>openapi.yaml</code>
1. Route	Which operation is this request?	Path and method under <code>paths</code>
2. Validation	Is the request shaped as promised?	<code>parameters</code> and <code>requestBody</code>
3. Work	What data is read or changed?	Domain rules and stored data
4. Representation	Which fields is the caller allowed to see?	Schema under <code>responses.*.content</code>
5. Response	Which status code?	Every key listed under <code>responses</code>

An operation is considered done only if all five parts exist **and** every status code listed under `responses` can actually be produced by some branch of the code. An operation that implements only the success path is not done, regardless of its line count.

A.3 List of operations and work order

Copy the list of operations from `openapi.yaml` into `service/README.md` and give each one a status. This table is updated throughout the session and is checked by the demonstrator.

Operation	Served by	Remaining work
<code>GET /v1/<resource>/{id}</code>	service	—
<code>GET /v1/<resource></code>	service	<code>status</code> filter not yet implemented
<code>POST /v1/<unsafe-collection></code>	mock	everything

Required order of work: **read operations before write operations, and a single entity before a collection.** The reason is that read operations let you inspect stored data without first having to trust the code that wrote it. Working on three operations at once is not allowed; one finished operation gives more information than three half-finished ones.

A.4 Implementing read operations

Implement at least one operation on a single entity and one operation on a collection.

Graded requirements:

- Validation happens before database access.** An identifier that doesn't match the schema's pattern produces `400`, not `404`. The two conditions are different statements: `400` means *the request could not be read as the contract promised*, while `404` means *the request was read, and that entity does not exist*. A client cannot tell a defect in itself apart from unavailable data if both produce the same status.
- Validation rules are derived from the schema in the contract.** Adding a rule the contract does not state, or skipping a rule the contract does state, is not allowed.
- Database rows are not returned directly.** Every resource has one explicit representation function. Two reasons are both graded: *exposure*, meaning the row carries columns the client should not see, such as

internal identifiers, audit columns, and soft-delete markers; and *coupling*, meaning the public interface becomes identical to the database schema of the moment, so a migration changes the interface without any reviewable specification change.

4. **Query parameters are part of the contract**, including their names, default values, and the maximum `limit` value. The pagination specified at Session 2 is implemented this session.
5. **An empty collection produces `200` with an empty array, not `404`**. A client that must handle two response shapes for one condition — *there is no data today* — carries two code paths for one situation.

A.5 Implementing write operations

Implement at least one write operation, namely the unsafe operation identified in the Session 2 assignment, section B.3.

Graded requirements:

1. **The body is validated once against the documented schema**, not through a scattered chain of `if` checks. A validation failure produces `400` with a list of invalid fields as an *extension member* on the Problem Details.
2. **The three failure statuses are distinguished consistently**, following the categories from the Session 2 assignment, section B.4.

Status	Server's claim	Example
<code>400</code>	The request could not be read as the contract promised.	Wrong data type; a required field missing; malformed JSON.
<code>422</code>	Every field is individually valid, but the request as a whole cannot be used.	An empty <code>lines</code> array; a referenced entity that does not exist.
<code>409</code>	The request is understood, but the current domain state rejects it.	Outlet is closed; stock is depleted; the status transition is not allowed.

Using `500` for any of these three conditions is graded incorrect. `500` means *the service failed while carrying out otherwise-valid work*, and the client handles it differently — by retrying. 3. **Domain rules are enforced in the service**. A client may anticipate the rules, for example by hiding a button, but that is a UX improvement, not a control mechanism. Every rule must be tested by sending a request directly with `curl`, without going through the client interface. 4. **A successful response returns `201`, a `Location` header, and a representation of the created entity** in the same shape a read operation on that entity would return. Values the server fills in — identifier, initial status, creation time, and totals — must be present in the response, so the client never has to guess them.

A.6 Cataloguing failures in code

1. **Every failure response is produced by a single function**, which sets `Content-Type: application/problem+json` and assembles the five RFC 9457 fields.
2. **Build a mapping table** and save it in `service/README.md`, with columns for **cause inside the handler**, **status code**, and **type URI**. Every row in that table must be listed in `openapi.yaml` on the operation that

can produce it. A mismatch between this table and the specification is one of the four defect classes checked at Session 7.

3. **There is one global error handler** that catches unexpected failures, logs the details, and returns `500` as Problem Details without any internal detail whatsoever. Stack traces, SQL fragments, internal hostnames, and database connection strings must not appear in the response body. The `instance` field links a user's report to the log entry.

A.7 Storage

1. **No significant data is kept in the process's memory.** This is checked with the following procedure: create three entities, stop the service process, start it again, then perform a read request. All three entities must still be there. This is a correctness issue before it is a scale issue; one process and one restart are already enough to show it.
2. **The database structure is stored in a committed file.** Any member must be able to build the database from an empty state with a single command, on a computer that has never been used for this before. "It works on my machine because I added a column manually" is the most common cause of demo failure at Session 7.
3. **Columns are derived from the documented representation,** plus whatever columns the service needs to operate. Column names are an internal matter; response field names are a contract matter. The representation function connects the two.
4. **All SQL lives inside `store/`.**

A.8 Server-side idempotency

The Session 2 assignment, section B.3, required the specification to state four idempotency-key rules. This session implements the server side of those four rules, following this decision order.

Condition	Server action	Response
Header missing or malformed	Reject before doing any work	<code>400</code>
Key never seen before	Record the key with a hash of the body, process the request, store the response against that key	<code>201</code>
Key already seen, identical body	Do not reprocess; resend the stored response	<code>201</code> with the same identifier
Key already seen, different body	Reject, because the key is being used for a different purpose	<code>409</code> <code>idempotency-key-reuse</code>

Rule on storing the key. The key is stored in a table, not in an in-process data structure. The reason is specific: the time window when a retry arrives is the time window when something is already going wrong, i.e. when the process is most likely to have just restarted. Storing the key in memory removes the protection mechanism exactly when it's needed. The body hash is stored alongside the key; without that hash, a client that reuses one key for two different requests silently loses the second request.

Verification procedure. Send the same write request twice with the same key, then compare the two responses and count the rows in the relevant table.

```
KEY=$(uuidgen)
for i in 1 2; do
  curl -s -o /tmp/r$i.json -w "%{http_code}\n" \
    -X POST "$BASE/v1/<your-unsafe-collection>" \
    -H "Idempotency-Key: $KEY" -H 'Content-Type: application/json' \
    -d '{ ... }'
done
diff /tmp/r1.json /tmp/r2.json && echo "identical responses"
```

Correct result: two 201 s, identical bodies, and exactly one new row. Two rows means the key was not checked. A second response of 409 means the implementation is wrong, because 409 is reserved for a different body with the same key.

A.9 Contract conformance testing

The mock server is correct by construction because it is generated from the specification. The service is not, so an automated check comparing the two is required.

1. **Run the contract test runner** the cohort agreed on against the running service. The check is run before an operation is declared done, not after the session ends.
2. **Run the same check on every push** via continuous integration. Five steps are enough: start the database, apply the database structure, start the service, wait for the service to be ready, and run the check. The value of this step is not the sophistication of the pipeline, but that it becomes impossible to merge a change that puts the service at odds with its own contract.
3. **Handling a failed check.** There are two legitimate fixes: fix the implementation, or deliberately revise the contract along with its version. Decide by asking which behaviour the client should be able to rely on. In most cases the implementation is wrong, because a 500 on invalid input tells the client to retry a request that will never succeed. Removing a field from the `required` list in `openapi.yaml` just to make the check pass is not a legitimate fix and is graded FAIL.

A.10 Configuration and deployment

1. **Three categories of values are distinguished:** *code*, which is committed and identical across all environments; *settings*, which differ between environments and are not secret; and *secrets*, which are supplied at process start and are never committed.
2. **service/.env.example is committed** and lists every variable name the service needs, with empty or example values. The `.env` file holding real values is listed in `.gitignore`.
3. **The service refuses to start if a required variable is missing.** A clear failure in the first second is far cheaper to diagnose than a confusing 500 an hour later.
4. **There is a /health endpoint** that returns 200 without checking any dependency. The platform uses it to decide whether to restart the process. A database check must not be placed there: a thirty-second database outage would make every instance report failure at once, causing the platform to restart all of them simultaneously and turning a brief outage into a long one.

5. **The service is reachable through a public URL.** Any hosting provider that fills the slot in Appendix C may be used. What is graded is not the choice of provider, but whether the steps in sections A.7 through A.10 can be repeated by someone else from a clean checkout.

A.11 Decision record

Create the file `docs/decisions/0002-implementasi.md` with four sections — **Context**, **Decision**, **Alternatives considered**, and **Consequences** — recording the choice of hosting provider, the idempotency-key storage mechanism, and any deviation from the directory structure in section A.1 along with the reason.

Part A grading criteria

Demonstrate the following three things to the demonstrator before the session ends: one read operation answered by the group's own code through the deployed URL; one write operation whose result can be found by a subsequent read operation; and the same write request sent twice with one key produces exactly one entity, which still exists after the service process is restarted.

Category	Criteria
PASS	All three demonstrated, no configuration values in source code, and the database structure is stored in a committed file.
PARTIAL	The service is deployed and works, but idempotency-key storage is in process memory, or configuration is hardcoded.
FAIL	Nothing is deployed outside a group member's own computer; or the service implementation code was committed earlier than the specification commit; or <code>openapi.yaml</code> was changed to make a failing test pass.

Mistakes to avoid

Mistake	Symptom	Fix
Database rows returned directly as the response	Internal columns appear in the response; renaming a column changes the public interface with no specification change	One explicit representation function per resource
Only the success path is written	A normal condition like "entity not found" reaches the client as <code>500</code>	One code branch for every documented response
<code>404</code> used for a malformed identifier	The client cannot tell a defect in itself apart from unavailable data	Validate first, then look up the data
The error shape is rebuilt in every handler	Three different error shapes exist in one API	One function that produces Problem Details
An uncaught exception becomes <code>500</code> with a stack trace	Internal paths and SQL fragments are visible to any caller	A global error handler; details go to the log, only <code>type</code> is returned

Mistake	Symptom	Fix
Data is stored in an in-process variable	All data is lost on every restart and every deployment	A table, with a committed <code>schema.sql</code>
The idempotency key is stored in memory	Duplicate entities appear precisely when the network is already having trouble	A table, checked before work is done
The database structure was created manually through a GUI	Other members cannot run the service from a clean checkout	Every <code>CREATE TABLE</code> lives in a committed file
Credentials are in the repository	Found by a grader via <code>git log</code> ; also a professional incident	Environment variables; rotate any value that was ever committed
<code>/health</code> checks the database	A brief database outage restarts every instance at once and hinders recovery	<code>/health</code> checks no external dependency at all
No contract test	The service and the spec drift apart with no signal until Session 7, when the drift costs marks	Contract tests in continuous integration from day one
The contract was changed to make the test pass	<code>openapi.yaml</code> 's history holds a change with no note in <code>CHANGELOG.md</code>	Fix the implementation; if the contract really is wrong, revise it deliberately with the reason

Format and submission

	Part A
Repository tag	13
Due	End of the Session 3 lab
Unit of work	Group
Additional deliverable	Deployment URL, listed in <code>README.md</code>
Verification	All three demonstrations in Part A's grading criteria performed live for the demonstrator before the session ends

```
git tag 13 && git push --tags
```

The tag uses a lowercase letter L followed by a number.

Relationship to A2

The code produced in this assignment becomes the basis for A2 — Backend Service, weighted at 15% of the final grade, due at Session 11 with an individual contribution statement.

Weight	A2 criterion	Part covered by this assignment
30%	Conformance — contract tests pass in continuous integration, including negative cases; no drift between the document and the behaviour	A.9
20%	Correctness — idempotency is correct under concurrent retries; status transitions are enforced; errors map to the documented catalogue	A.5, A.6, A.8
20%	Operability — deployable from a clean checkout by someone who has never met its author; configuration is validated at startup; migrations are forward-only	A.7, A.10
15%	Authorisation — scope and object-ownership checks in every handler that references an object	Session 4
15%	Observability — structured logging; correlation identifier; a real failure can be diagnosed directly within two minutes	Session 9

Seventy percent of A2's weight comes from Session 3's material. The rest is added at Session 4 and Session 9 on top of the foundation built today.

Self-check questions

1. A handler returns `404` both when an identifier doesn't match the schema's pattern and when the entity is not found. Describe two different consequences the client experiences from merging these two conditions.
2. Give two distinct reasons for never returning a database row directly as a response body.
3. A contract test reports `500` on an operation that, per the specification, should return `422`. State two possible fixes and explain the basis for choosing between them.
4. An idempotency key is stored in an in-process data structure. Describe the complete sequence of events that ends with a user being charged twice, with only one service instance running.
5. A group member proposes that `/health` check the database connection "so we know the service is actually working." Explain the disruption this proposal causes and name the right place for that check.
6. A write operation returns `200` with the body `{"created": true}`. Name three things missing compared to a correct response, and explain the extra work the client must do because of each.